

제138회 컴퓨터시스템응용기술사 해설집

2026.02.07

국가기술자격 기술사 시험문제

기술사 제 138 회

제 2 교시 (시험시간: 100 분)

분야	정보통신	자격종목	컴퓨터시스템응용기술사	수점 번호		성 명	
----	------	------	-------------	----------	--	--------	--

※ 다음 문제 중 4 문제를 선택하여 설명하십시오. (각 10 점)

1. 뉴로모픽 컴퓨팅(Neuromorphic Computing)과 기존 폰 노이만 아키텍처에 대하여 다음 사항을 설명하십시오.

가. 뉴로모픽 컴퓨팅의 개념

나. 폰 노이만 아키텍처와의 비교

다. SNN(Spiking Neural Network) 동작 원리

라. 활용 분야

2. 대규모 언어 모델(LLM)이 서버·클라우드·엣지 환경에 적용되면서 새로운 보안 및 운영 문제가 발생하고 있다. OWASP의 LLM 보안 위협을 중심으로 다음 사항을 설명하십시오.

가. LLM이 컴퓨터시스템 자원에 미치는 영향

나. 주요 위협과 시스템 수준의 발생 원인

다. LLM 보안 위협에 대한 대응방안

3. ISO/PAS 8800:2024에서 제시하는 AI 안전 확보에 대한 다음 사항을 설명하십시오.

가. 생명주기 관점에서의 주요 요구사항

나. ISO 26262 및 ISO 21448과의 연계 확장 관계

다. 자율주행 기능에 적용한 사례

4. Benchmark Test, PoC(Proof of Concept), Pilot Test 의 차이점을 비교하고, 프로젝트 수행 시 통합 적용하는 시나리오를 제시하시오.
5. 운영체제의 프로세스 스케줄링에 대하여 다음 사항을 설명 하시오.
- 가. 목적
 - 나. 주요 스케줄링 알고리즘
6. 명령어 실행하기 위해 필요한 컴퓨터 CPU 내부 레지스터 종류들에 대하여 설명하시오.

01	뉴로모픽 컴퓨팅(Neuromorphic Computing)		
문제	뉴로모픽 컴퓨팅(Neuromorphic Computing)과 기존 폰 노이만 아키텍처에 대하여 다음 사항을 설명하시오. 가. 뉴로모픽 컴퓨팅의 개념 나. 폰 노이만 아키텍처와의 비교 다. SNN(Spiking Neural Network) 동작 원리 라. 활용 분야		
도메인	인공지능	난이도	중 (상/중/하)
키워드	뇌 모사, 이벤트 기반 처리, 인메모리 컴퓨팅, 초저전력, 대규모 병렬성		
출제배경	폰 노이만 한계를 극복하기 위한 뉴로모픽 컴퓨팅과 SNN 기반 차세대 AI 아키텍처에 대한 이해 확인		
참고문헌	ITPE 서브노트		
출제자	강남평일야간반 전일 기술사(제 114회 정보관리기술사 / nikki6@hanmail.net)		

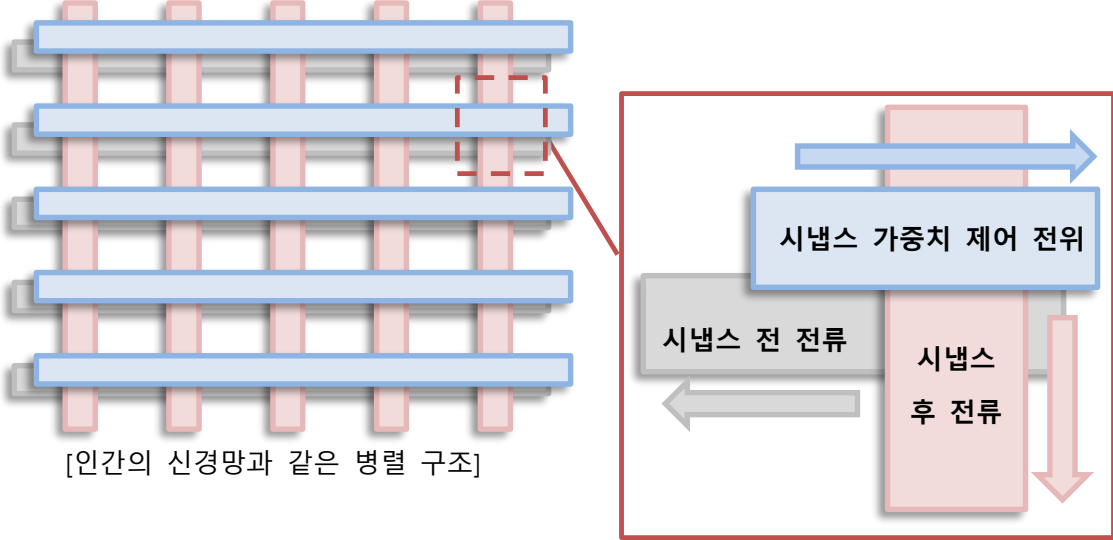
I. 폰노이만 아키텍처의 한계

관점	한계	설명
성능관점	- 공통 버스 사용	- 명령어와 데이터가 동일한 버스 사용. 병목발생
	- 메모리 접근 지연	- CPU와 메모리간 속도 차이로 전체 속도가 제한
보안관점	- 명령어/데이터 구분 부족	- 코드와 명령어가 같은 메모리에 존재, 악성코드 전이 가능
	- 권한 분리 미흡	- 초기 구조는 사용자와 커널모드의 명확한 분리 미고려
에너지효율 관점	- 불필요한 데이터 이동	- CPU와 메모리 간의 빈번한 데이터 이동
	- 캐시 미스 비용	- 캐시 계층이 깊어질수록 미스 발생시 에너지 소모
병렬성 관점	- 직렬 처리 기반	- 순차 명령 실행을 전제, 병렬처리에 비효율
	- 멀티코어 확장 어려움	- 구조적으로 병렬성을 미고려, 멀티코어에서 병목 발생
확장성 및 현대 응용 관점	- AI/머신러닝 비효율	- 대규모 행렬 연산이나 병렬 처리가 중요한 AI 분야에서는 비효율적
	- 메모리 중심 구조 한계	- 데이터 중심 컴퓨팅(DPU, PIM 등)과 같은 새로운 패러다임과는 맞지 않는 구조

- 폰노이만 아키텍처는 대규모 행렬 연산이나 병렬 처리가 중요한 AI분야에서 비효율적

II. 뉴로모픽 컴퓨팅의 개념과 폰 노이만 아키텍처와의 비교

가. 뉴로모픽 컴퓨팅의 개념

구분	설명	
개념	- 인간의 뇌 신경망 구조(뉴런과 시냅스)를 하드웨어적으로 모방한 비 폰노이만형 컴퓨팅 구조	
개념도	 [인간의 신경망과 같은 병렬 구조]	
구성요소	- 뉴런 회로	- 입력된 스파이크 신호를 누적·처리하고 임계값 초과시 출력 스파이크를 발생
	- 시냅스 회로	- 뉴런 간 연결 강도(가중치)를 조절하며, 학습을 담당
	- 스파이크 신호	- 전류 펄스 형태로 정보 전달. 디지털이 아닌 이벤트기반 통신
	- 메모리 소자	- 시냅스 가중치를 저장하고 가변저항 형태로 학습 반영
	- 학습 및 제어 회로	- STDP(Spike-Timing Dependent Plasticity) 등 학습규칙 적용
	- 뉴런 어레이	- 뉴런/시냅스의 대규모 병렬 집합체. 뇌신경망 유사구조 형성
	- 입출력 인터페이스	- 외부와의 상호작용, 영상·음성·센서 데이터입력 및 결과 출력
	- 통신 인터커넥트	- 뉴런 간 스파이크 신호를 연결하는 네트워크 구조

- 인간의 뇌를 모사한 뉴런(연산단위), 시냅스(학습 및 기억), 스파이크 신호(통신방식), 이를 지탱하는 메모리 소자와 학습회로, 그리고 입출력 인터페이스로 구성되며, 이들이 대규모 병렬 어레이로 연결되어 뇌 신경망 유사 구조를 형성

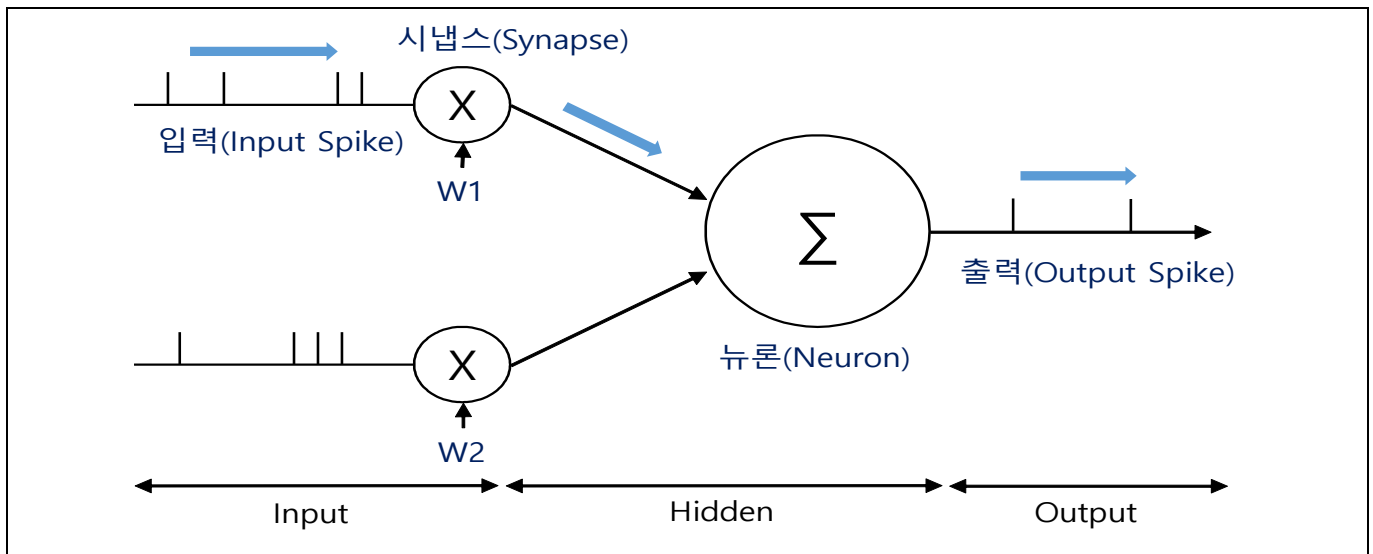
나. 폰 노이만 아키텍처와의 비교

구분	뉴로모픽 컴퓨팅	폰 노이만 아키텍처
기본 개념	- 인간 뇌 신경 구조를 모방한 컴퓨팅	- CPU-메모리 분리 기반 전통적 구조
처리 방식	- 이벤트 기반, 비동기 처리	- 명령어 기반, 동기식 처리
연산 모델	- 스파이킹 뉴런, 병렬·분산 처리	- 순차 처리 중심
메모리 구조	- 연산·저장 통합	- 연산(CPU)과 저장(Memory) 분리
병목 현상	- 병목 거의 없음	- 폰 노이만 병목 발생
전력 효율	- 매우 높음(저전력)	- 상대적으로 전력 소모 큼

병렬성	- 대규모 병렬 처리에 최적	- 제한적 병렬 처리
학습·적응	- 실시간 학습·적응 가능	- 사전 학습 모델 실행 중심
처리 대상	- 패턴 인식, 감각 정보, AI 추론	- 범용 연산, 데이터 처리
활용 분야	- 로봇, 자율주행, 엣지 AI	- 서버, PC, 범용 컴퓨팅
성숙도	- 연구·초기 상용 단계	- 완전 성숙 기술

III. SNN(Spiking Neural Network) 동작 원리

가. SNN의 구조 개념도



- 두뇌를 구성하는 뉴런과 시냅스를 인공지능 구현 형태로 추상화한 LIF(Leaky Integrated-and-Fire) 모델

나. SNN의 동작원리

단계	동작 원리	핵심 설명	키워드
① 입력	- 스파이크 입력	- 연속값이 아닌 이산적 스파이크 로 정보 전달	- Spike
② 적분	- 막전위 누적	- 입력 스파이크를 시냅스 가중치에 따라 막전위로 적분	- Integrate
③ 임계값 판단	- 발화 조건 판단	- 막전위가 임계값(Threshold) 초과 여부 판단	- Threshold
④ 발화	- 스파이크 출력	- 임계값 초과 시 스�파이크 1 회 발화	- Fire
⑤ 리셋	- 상태 초기화	- 발화 후 막전위를 초기 상태로 Reset	- Reset
⑥ 처리 방식	- 비동기·병렬	- 글로벌 클럭 없이 이벤트 기반 병렬 처리	- Event-driven
⑦ 학습	- 시간 기반 학습	- 스파이크 **시간 차이(STDP)** 에 따라 가중치 조정	- STDP

- SNN 은 뉴런의 막전위 누적과 임계값 발화를 기반으로 스파이크를 생성하며, 시간 정보를 직접 활용하는 이벤트 기반 신경망 구조

IV. 뉴로모픽 컴퓨팅 활용 분야

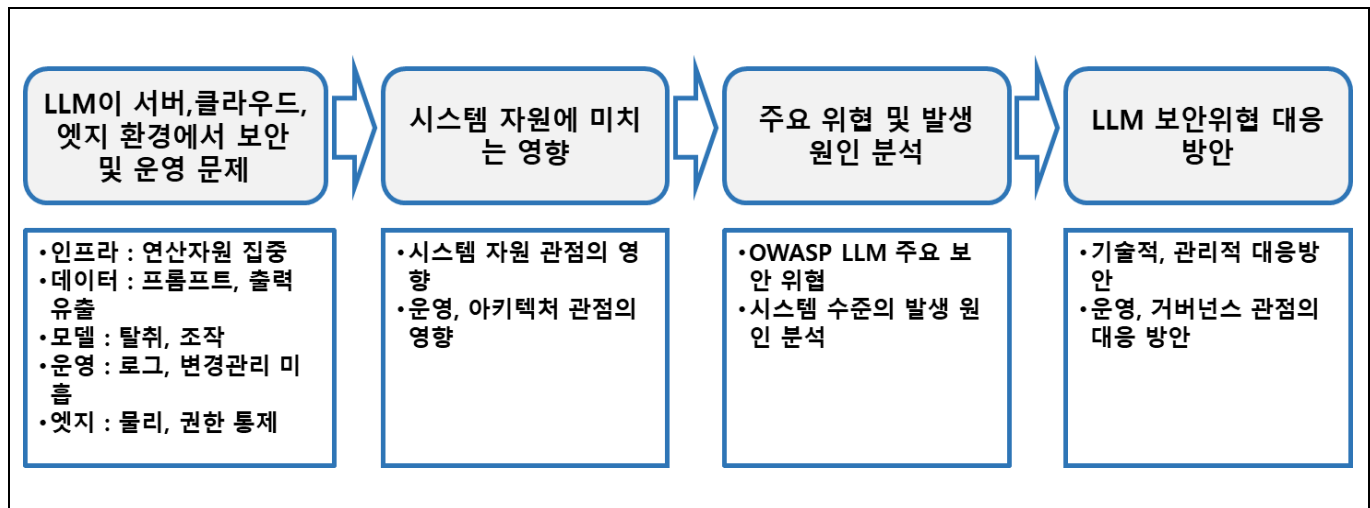
분야	활용 내용	활용 이유(강점)
엣지 AI	- 센서 데이터 실시간 분석	- 저전력·이벤트 기반 처리로 배터리 효율 우수
로봇·지능형 기기	- 자율 행동, 환경 인지	- 실시간 반응·병렬 처리에 유리
자율주행	- 객체 인식, 상황 판단	- 저지연·고속 인지 처리
영상·시각 처리	- 이벤트 카메라 기반 영상 분석	- 불필요 연산 제거, 초저전력
음성·청각 인식	- 소리 패턴 인식, 음성 처리	- 시간 정보 기반 인식에 강점
IoT·스마트 센서	- 이상 탐지, 패턴 감지	- 센서 근접 처리로 통신·전력 절감
뇌과학·의료	- 신경 신호 분석, 뇌 시뮬레이션	- 생물학적 신경 구조 모사

- 뉴로모픽 컴퓨팅은 저전력·비동기·고병렬 특성을 바탕으로 엣지 AI, 로봇, 자율주행, 실시간 인지·감각 처리 분야에 활용

“끝”

02	대규모 언어 모델(LLM)의 보안 및 운영 문제		
문제	대규모 언어 모델(LLM)이 서버·클라우드·엣지 환경에 적용되면서 새로운 보안 및 운영 문제가 발생하고 있다. OWASP의 LLM 보안 위협을 중심으로 다음 사항을 설명하시오. 가. LLM이 컴퓨터시스템 자원에 미치는 영향 나. 주요 위협과 시스템 수준의 발생 원인 다. LLM 보안 위협에 대한 대응방안		
도메인	인공지능(AI)	난이도	상 (상/중/하)
키워드	OWASP LLM Top Threat, Prompt Injection 공격, 자원 고갈 및 LLM DoS, AI 거버넌스 및 DevSecOps		
출제배경	OWASP LLM 보안 위협을 기반으로 한 시스템 수준의 대응 전략을 제시		
참고문헌	OWASP LLM Top Threat, 국내 AI 보안 가이드, ITPE 서브노트		
출제자	정주행 조종흥 기술사(제127회 정보관리기술사 / jongheung.cho@gmail.com)		

I. LLM 확산에 따른 컴퓨팅 환경 변화와 문제에 대한 개요



- LLM 은 서버·클라우드·엣지 전반에서 자원·데이터·모델·운영 영역의 새로운 보안 위협을 유발하므로, 시스템 수준의 원인 분석과 기술·거버넌스 기반의 종합적 대응이 필요.
- 전통적 애플리케이션 보안 모델은 LLM 의 프롬프트·모델·추론 특성을 충분히 반영하지 못하므로, OWASP LLM 보안 위협에 대한 체계적 논의가 필요

II. LLM 이 컴퓨터 시스템 자원에 미치는 영향

가. 시스템 자원 관점의 영향

구분	영향 요소	설명
연산 자원	CPU·GPU 고집중 사용	대규모 행렬 연산 및 병렬 처리로 인해 GPU 중심의 연산 자원 집중 발생
메모리	메모리 사용량 급증	장문 프롬프트·컨텍스트 유지로 RAM 및 GPU 메모리 소모 증가
스토리지	I/O 부하 증가	모델 로딩, 캐시, 로그 저장 등으로 디스크 접근 빈도 증가

성능	처리 지연 및 병목	동시 추론 요청 증가 시 대기 시간 증가 및 처리량 저하
가용성	자원 고갈 위험	악의적 요청 또는 트래픽 폭증 시 서비스 거부(DoS) 가능성 증가

- LLM 은 대규모 연산과 컨텍스트 처리를 통해 CPU·GPU·메모리 자원을 고집중적으로 소모하여 성능 저하와 자원 고갈 위험을 증대시킴.

나. 운영·아키텍처 관점의 영향

구분	영향 요소	설명
비용	클라우드 비용 급증	GPU 사용량 증가로 사용량 기반 비용 예측 및 통제 곤란
확장성	오토스케일링 복잡성	추론 지연·콜드 스타트 등으로 단순 스케일링 정책 적용 한계
안정성	서비스 품질 변동	요청 패턴 변화에 따라 응답 지연 및 품질 편차 발생
엣지 환경	자원 제약 문제	제한된 연산·전력 환경에서 모델 실행 시 안정성 저하
운영 관리	운영 난이도 증가	자원 모니터링, 성능 튜닝, 장애 원인 분석 복잡화

- LM 도입은 클라우드 비용 증가와 확장성·안정성 관리 복잡성을 야기하며, 특히 엣지 환경에서는 자원 제약으로 서비스 운영 난이도를 높임.

III. OWASP LLM 주요 보안 위협과 시스템 수준의 발생 원인

가. OWASP LLM 주요 보안 위협

구분	주요 위협	설명
입력 기반 공격	Prompt Injection / Indirect Prompt Injection	사용자 입력 또는 외부 콘텐츠를 통해 시스템 프롬프트를 무력화하거나 의도하지 않은 명령 수행 유도
자원 오남용	모델 오용(Model Abuse)	반복·대량 추론 요청을 통해 모델 자원을 과도하게 소비하는 행위
가용성 위협	서비스 거부 공격 (LLM DoS)	고비용 추론 요청으로 CPU·GPU 자원 고갈을 유발하여 정상 서비스 제공 방해
정보 유출	민감정보 노출	모델 출력 또는 로그를 통해 개인정보·기밀정보가 노출될 위험
지식 추론	학습 데이터 추론	반복 질의를 통해 학습 데이터의 일부를 역으로 추론 가능
권한 침해	권한 우회	프롬프트 조작을 통해 비인가 기능이나 시스템 자원 접근 시도

- OWASP 는 LLM 환경에서 입력 기반 공격, 자원 오남용, 정보 유출, 권한 우회 등 기존 보안 모델로는 대응이 어려운 복합적 위협이 발생함을 지적

나. 시스템 수준의 발생 원인 분석

구분	발생 원인	설명
설계 관점	블랙박스 기반 LLM 활용	내부 동작과 판단 근거가 불투명하여 보안 통제 지점 식별 및 검증이 어려움

통제 관점	입력·출력 검증 미흡	프롬프트 및 응답에 대한 필터링·정책 검증 부재로 공격 입력이 그대로 처리됨
권한 관리	권한·컨텍스트 통제 부재	LLM 요청 범위와 실행 권한이 명확히 분리되지 않아 오남용 가능
연계 구조	기존 시스템과 과도한 결합	API·DB·에이전트와 직접 연동되어 단일 취약점이 전체 시스템으로 확산
운영 구조	보안 책임 분산	LLM·플랫폼·업무 시스템 간 보안 책임 경계가 불명확

- 이러한 위협은 LLM 을 블랙박스로 설계하고 입력·권한·연계 통제를 충분히 고려하지 않은 시스템 아키텍처에서 구조적으로 발생

IV. LLM 보안 위협에 대한 대응 방안

구분	대응 방안	설명
기술적, 관리적 대응	입력/출력 검증 및 프롬프트 가드레일	악의적 프롬프트 및 비정상 출력 차단을 위한 정책 기반 필터링 적용
	보안 정책 및 룰 기반 제어	허용된 요청 유형·범위만 처리하도록 사전 정의된 보안 정책 적용
	권한 분리 및 모델 접근 통제	LLM, API, 에이전트 간 최소 권한 원칙 적용으로 오남용 방지
	자원 사용 제한(Rate Limit)	추론 요청 빈도·비용 제한을 통해 LLM DoS 및 자원 고갈 방지
	보안 모니터링 강화	추론 패턴·이상 요청 탐지를 통한 실시간 위협 식별
운영, 거버넌스 관점의 대응	감사 로그 관리	프롬프트·응답·접근 이력에 대한 추적성 확보로 사고 분석 지원
	OWASP LLM Top Threat 기반 설계	OWASP LLM 위협 모델을 반영한 보안 아키텍처 체계화
	DevSecOps 및 AI 거버넌스 연계	개발·배포·운영 전 과정에 보안 통제 및 책임 체계 내재화
	평가·개선 체계 확립	모델 변경·환경 변화에 따른 정기적 보안 점검 및 개선 수행

- LLM 보안 위협 대응은 입력·권한·자원 통제와 모니터링을 기술적으로 구현하고, OWASP LLM 위협 모델과 DevSecOps·AI 거버넌스를 연계한 지속적 운영 체계로 완성되어야 함.

“끝”

03	ISO/PAS 8800:2024 AI 안전 확보		
문제	ISO/PAS 8800:2024에서 제시하는 AI 안전 확보에 대한 다음 사항을 설명하시오. 가. 생명주기 관점에서의 주요 요구사항 나. ISO 26262 및 ISO 21448과의 연계 확장 관계 다. 자율주행 기능에 적용한 사례		
도메인	인공지능(AI)	난이도	상 (상/중/하)
키워드	ISO 26262 : 고장 기반 안전, ISO 21448 : 정상 동작 중 기능 한계, ISO/PAS 8800 : 데이터·학습·운영 변화로 인한 AI 특유 위험		
출제배경	AI 안전 확보를 위한 생명주기 기반 요구사항, 기존 안전 표준과의 연계 확장 구조, 자율주행 적용 사례에 대한 이해		
참고문헌	(AI Safety) 자율주행 AI의 안전 확보를 위한 새로운 표준: ISO/PAS 8800:2024		
출제자	정주행 조종흥 기술사(제127회 정보관리기술사 / jongheung.cho@gmail.com)		

I. AI 안전 중심 표준, ISO/PAS 8800:2024 의 개요

개념	AI 구성요소를 포함한 시스템에서 발생 가능한 안전 위험을 체계적으로 관리하기 위한 AI 안전 가이드라인으로, 기존 기능 안전 표준을 AI 특성에 맞게 확장한 표준	
구분	특징	설명
AI 안전 개념의 확장 필요성	안전 개념의 확장 요구	- 규칙 기반 제어에서 데이터 기반·확률 기반 AI로 전환 - 시스템 동작의 비결정성, 내부 판단 과정의 비가시성, 운영 중 성능 변화 등 새로운 안전 리스크가 증가함
기존 기능 안전 표준의 한계와 AI 안전 요구의 등장	기존 안전 표준의 적용 한계	- ISO 26262는 하드웨어·소프트웨어 고장 중심 - ISO 21448은 의도된 기능의 안전에 초점을 둠. - AI의 학습·추론 오류, 데이터 편향, 모델 변화와 같은 AI 고유 위험을 충분히 다루지 못함
역할 및 적용 범위	생명주기 전반 안전 관리	- AI 기반 시스템을 대상으로 기획, 개발, 검증, 운영 전 과정에서 AI 안전 확보를 위한 요구사항과 - 관리 원칙을 제시하는 보완·확장 표준으로, 자율주행 등 고위험 AI 시스템에 적용 가능함

- AI 기반 시스템의 확산은 기존 기능 안전 개념을 넘어서는 새로운 안전 체계의 정립을 요구하고 있음.

II. 생명주기 관점에서의 주요 요구사항

단계	주요 요구사항	설명
목표 및 범위 정의 (Concept Phase)	ODD(운행 가능 영역) 정의	AI가 정상적으로 작동해야 하는 환경 조건(날씨, 도로 유형, 조도 등)을 명확히 정의하여 적용 범위와 한계를 설정함
	안전 목표 설정	AI 오작동 시 발생 가능한 위험원(Hazards)을 식별하고, ASIL 수준에 따라 달성해야 할 안전 목표를 수립함

데이터 준비 및 관리 (Data Acquisition & Preparation)	데이터 완전성·다양성 확보	ODD 내 발생 가능한 다양한 시나리오가 학습 데이터에 충분히 포함되었는지 검증하여 미탐·오탐 위험을 최소화함
	데이터 어노테이션 품질 관리	레이블링 기준의 일관성과 정확성을 확보하고, 필요 시 독립적 검수 절차를 통해 데이터 신뢰성을 강화함
	데이터 편향성 관리	특정 상황(예: 야간, 악천후)에 데이터가 편중되지 않도록 균형을 유지하여 판단 왜곡을 방지함
모델 설계 및 학습 (Model Design & Training)	아키텍처 적절성 검증	선택한 신경망 구조가 설정된 안전 목표를 충족할 수 있음을 기술적·논리적으로 입증함
	강건성(Robustness) 확보	노이즈, 입력 변형 등 외란에 대해 출력이 급격히 변하지 않도록 정규화, 데이터 증강 등 학습 전략을 적용함
검증 및 타당성 확인 (Verification & Validation)	정적 분석 수행	모델 구조 및 설계 상의 잠재적 취약점을 사전에 분석하여 구조적 위험을 식별함
	동적 분석(테스트) 수행	학습에 사용되지 않은 독립 테스트셋을 활용하여 성능과 안전성을 객관적으로 검증함
	불확실성 영역 식별	AI가 판단에 확신을 가지지 못하는 영역을 정량화하여 한계 조건으로 관리함
통합 및 차량 실장 (Integration & Implementation)	HW/SW 인터페이스 안전성 확보	GPU·NPU 등 가속기와의 인터페이스에서 발생 가능한 오류를 식별하고 안전성을 검증함
	실시간성 보장	차량 내 제한된 자원 환경에서 AI 추론이 최악 실행 시간(WCET) 내에 완료됨을 검증함
사후 관리 및 모니터링 (Post-deployment Monitoring)	성능 저하 감시	실제 운행 중 데이터 분포 변화(Distribution Shift)를 탐지하고, 위험 발생 시 Fail-safe 조치를 수행함
	지속적 학습 통제	배포 이후 재학습·모델 업데이트 시 기존 안전성을 훼손하지 않도록 재검증 절차를 수행함

- ISO/PAS 8800은 AI 시스템을 개발 산출물이 아닌 운영 중에도 위험이 변화하는 진화형 시스템으로 정의하고, 데이터·모델·운영 전 단계를 포괄하는 생명주기 기반 안전 요구사항을 제시함.

III. SO 26262 및 ISO 21448과의 연계 확장 관계

가. 기능 안전 관점의 ISO 26262와의 관계

구분	내용
표준 개요	ISO 26262는 자동차 전기·전자 시스템의 하드웨어·소프트웨어 고장을 중심으로 기능 안전을 규정하는 표준
안전 접근 방식	고장(Fault) → 오류(Error) → 사고(Hazard)로 이어지는 고장 기반(Function Safety) 위험 분석
AI 적용 시 한계	AI는 하드웨어·소프트웨어 고장이 없더라도 학습 오류, 데이터 편향, 비결정성으로 인해 오동작 가능

확장 필요성	전통적 고장 중심 안전만으로는 AI 판단 오류 및 학습 기반 위험을 충분히 통제하기 어려움
ISO/PAS 8800 연계	ISO/PAS 8800 은 AI 의 비결정성·학습 특성을 고려한 안전 요구사항을 추가하여 ISO 26262 의 기능 안전 개념을 AI 영역까지 확장함

- ISO/PAS 8800 은 ISO 26262 의 고장 기반 기능 안전을 AI 비결정성·학습 특성까지 확장함.

나. 의도된 기능 안전 관점의 ISO 21448(SOTIF)과의 관계

구분	내용
표준 개요	ISO 21448 은 시스템 고장이 없더라도 의도된 기능의 한계로 인해 발생하는 위험(SOTIF)을 다루는 표준
안전 접근 방식	정상 동작 상태에서의 인지 부족, 환경 인식 한계, 사용 시나리오 미고려에 따른 위험 관리
AI 와의 연관성	자율주행 AI 의 인지·판단·환경 인식 오류는 SOTIF 위험의 대표 사례
기존 한계	AI 모델의 학습 과정, 데이터 편향, 모델 변화에 대한 구체적 안전 요구는 상대적으로 제한적
ISO/PAS 8800 연계	ISO/PAS 8800 은 AI 인식·판단 모델의 학습, 검증, 운영 단계까지 포함하여 SOTIF 개념을 AI 생명주기 전반으로 확장함

- ISO 21448 은 고장 없는 상태에서도 발생하는 의도된 기능의 한계 위험을 다루며, ISO/PAS 8800 은 이를 AI 인식·판단 모델의 학습·검증·운영 전 생명주기로 확장함.

다. 3 개 표준의 상호 보완적 적용 구조

구분	ISO 26262	ISO 21448 (SOTIF)	ISO/PAS 8800
안전 관점	고장 기반 안전	의도된 기능의 한계 안전	AI 특화 안전
주요 대상	HW/SW 결함	인지·환경 인식 한계	데이터·학습·모델
위험 원인	하드웨어 고장, 소프트웨어 오류	정상 동작 중 판단 부족	비결정성, 데이터 편향, 모델 변화
적용 범위	전통적 제어 시스템	인지·판단 기능	AI 생명주기 전반
확장 구조	기본 안전 프레임	기능 한계 보완	수직·수평 확장 레이어
상호 관계	기본 토대	기능적 보완	AI 안전 공백 해소

- ISO/PAS 8800 은 ISO 26262 와 ISO 21448 을 기반으로 AI 안전을 생명주기 전반으로 확장하는 상위 보완 표준

IV. 자율주행 기능에 적용한 사례

표준	사례	특징	설명
ISO 26262	브레이크 제어 ECU 이중화	하드웨어 고장 대응	제동 ECU 고장 시에도 안전 기능을 유지할 수 있도록 이중화 구조를 적용하여 단일 고장으로 인한 사고를 방지함
	센서 신호 단선·오류 감지	고장 진단 메커니즘	카메라·레이더 신호 단선 또는 이상 발생 시 이를 즉시 감지하고 시스템을 안전 모드로 전환함

	제동 액추에이터 실패 대응	ASIL 기반 안전 설계	제동 명령 미수행 시 비상 제동 또는 운전자 경고를 수행하도록 ASIL 수준에 따라 안전 메커니즘을 설계함
ISO 21448	역광 환경 보행자 인식 실패	정상 동작 중 인지 한계	센서와 시스템에 고장이 없더라도 강한 역광으로 인해 보행자를 인식하지 못하는 SOTIF 위험을 식별·분석함
	도로 공사 구간 차선 오인식	환경 인식 한계	임시 차선, 페인트 마모 등 비정형 도로 환경에서 AI의 차선 인식 오류 가능성을 관리함
	비표준 교차로 주행 판단 오류	사용 시나리오 미고려	설계 시 고려되지 않은 복합 교차로에서 AI 주행 판단이 불완전해질 수 있는 위험을 식별함
ISO/PAS 8800	학습 데이터 편향으로 인한 미탐	데이터 기반 위험	주간·맑은 날 데이터 위주 학습으로 야간·우천 환경에서 보행자 미인식 위험이 발생함
	운영 중 데이터 분포 변화 대응	성능 열화 관리	실제 주행 환경이 학습 데이터와 달라질 경우 성능 저하를 탐지하고 Fail-safe 동작을 수행함
	재학습·OTA 업데이트 안전 통제	AI 생명주기 관리	모델 재학습 및 OTA 업데이트 시 기존 안전성을 훼손하지 않도록 재검증 절차를 수행함

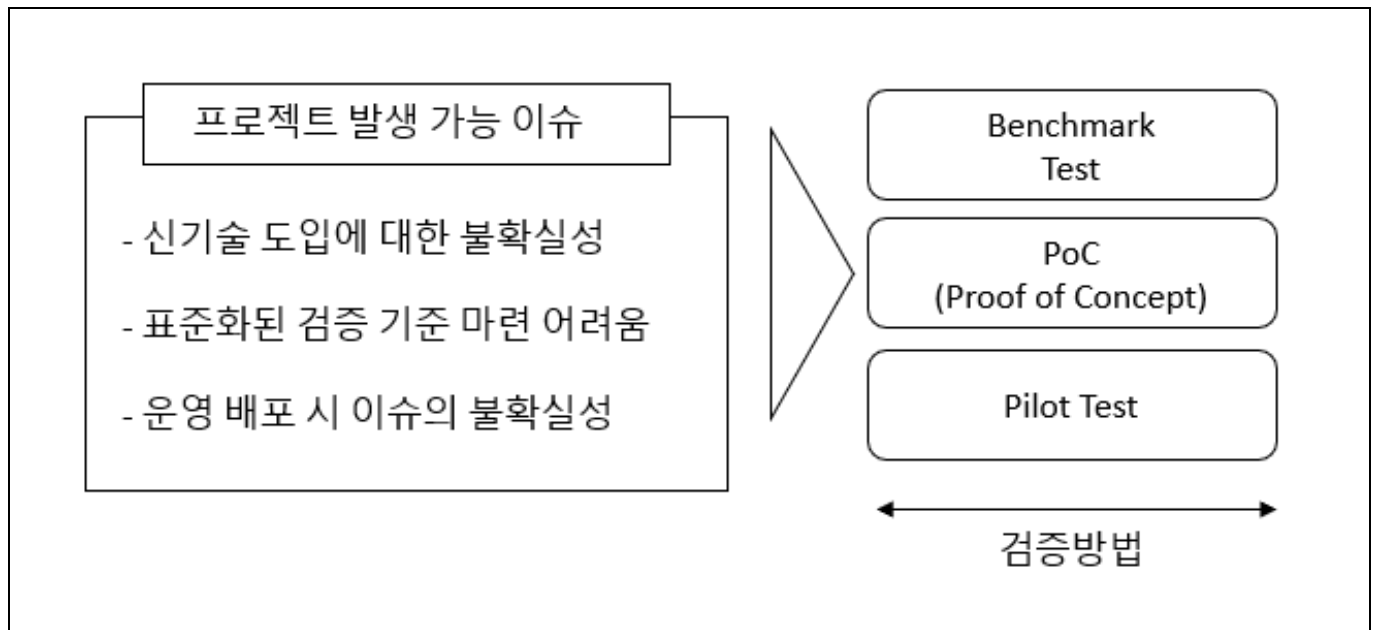
- ISO 26262는 고장 기반 안전, ISO 21448은 정상 동작 중 기능 한계, ISO/PAS 8800은 데이터·학습·운영 변화로 인한 AI 특유 위험을 각각 담당하며 상호 보완적으로 자율주행 안전을 완성

- ISO/PAS 8800은 AI 중심 시스템에서 안전 개념을 확장하고, ISO 26262와 ISO 21448을 실질적으로 보완하여 향후 자율주행 및 고위험 AI 시스템의 안전 확보를 위한 핵심 기준 역할을 수행

“끝”

04	Benchmark Test, PoC(Proof of Concept), Pilot Test		
문제	Benchmark Test, PoC(Proof of Concept), Pilot Test의 차이점을 비교하고, 프로젝트 수행 시 통합 적용하는 시나리오를 제시하시오.		
도메인	소프트웨어공학	난이도	중 (상/중/하)
키워드	기술 타당성, 핵심 가설 검증, 최소 구현(프로토타입), 데모, 현장 적용, 실제/준실 운영, 사용자-업무 시나리오, KPI(효과) 검증		
출제배경	프로젝트 신기술 도입을 위한 검증 기법 이해 확인 (최근 AX로 신기술 도입 사례 증가)		
참고문헌	ITPE 서브노트		
출제자	NS반 백현 기술사(제 122회 정보관리기술사 / snuoo@naver.com)		

I. 프로젝트 리스크 최소화 및 객관성 확보를 위한 검증 기법의 중요성



- IT 프로젝트는 복잡성이 높고 도입 기술의 불확실성이 크기 때문에, 대규모 예산 투입 전 리스크를 최소화 및 의사결정의 객관성을 확보하는 검증 절차 진행이 필수
- PoC, BMT, Pilot 은 프로젝트 SDLC 초기 단계에서 수행되는 핵심적인 검증 활동

II. Benchmark Test, PoC, Pilot Test 의 차이점 비교

가. Benchmark Test, PoC, Pilot Test 개념 및 목적 측면의 차이점 비교

비교	Benchmark Test,	PoC	Pilot Test
개념	- 표준화된 테스트를 통한 성능 비교	- 새로운 기술/아이디어의 실현 가능성 검증	- 실제 환경 적용 전 소규모 운영 테스트
목적	- 제품 간 비교 우위 산정	- 기술 타당성 확인	- 운영 리스크 사전 탐지
수행시점	- 제품 도입 및 기획 단계	- 설계 및 구현 초기 단계	- 운영 전환 직전 단계
의사결정	- 벤더 선정의 근거	- 기술 채택 여부 결정	- 전사 확산 여부 판별
성공기준	- 목표치 대비 성능 우위	- 기술 가설 입증	- KPI 달성 + 운영 조건 충족

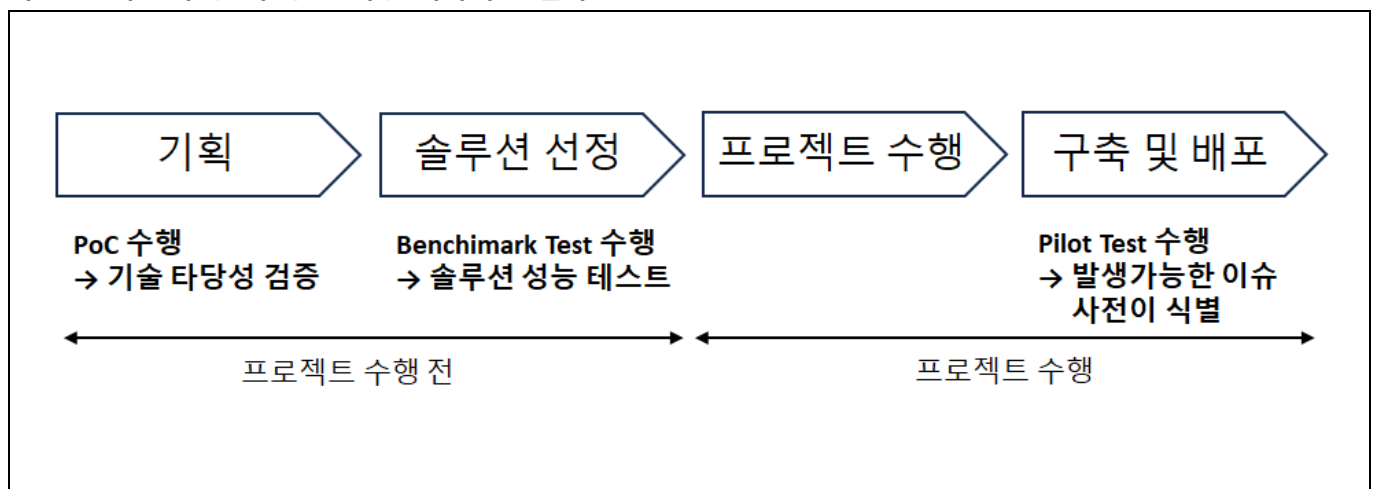
개념도			
	성공기준	- 목표치 대비 성능 우위	- 기술 가설 입증
	주요관점	- 효율성	- 타당성
	대상	- 솔루션 간 상대평가	- 단일 기술의 절대 평가
		- KPI 달성 + 운영 조건 충족	- 안정성

나. Benchmark Test, PoC, Pilot Test 수행 및 산출물 측면의 비교

비교	Benchmark Test,	PoC	Pilot Test
수행주체	- 발주처 & 제안사	- 아키텍트	- 프로젝트팀 + 현업사용자
테스트 환경	- 테스트 베드	- 샌드박스	- 실 운영환경
테스트 기준	- 목표 기준 달성 여부	- 기술 구현 가능 여부	- 업무 프로세스 적합성
데이터	- 표준화된 더미 데이터	- 샘플/가공 가능	- 운영 데이터 중심
산출물	- 결과 비교표	- 기술 채택 여부 결정	- 전사 확산 여부 판별

III. 프로젝트 수행 시 통합 적용 시나리오

가. 프로젝트 수행 시 통합 적용 시나리오 절차



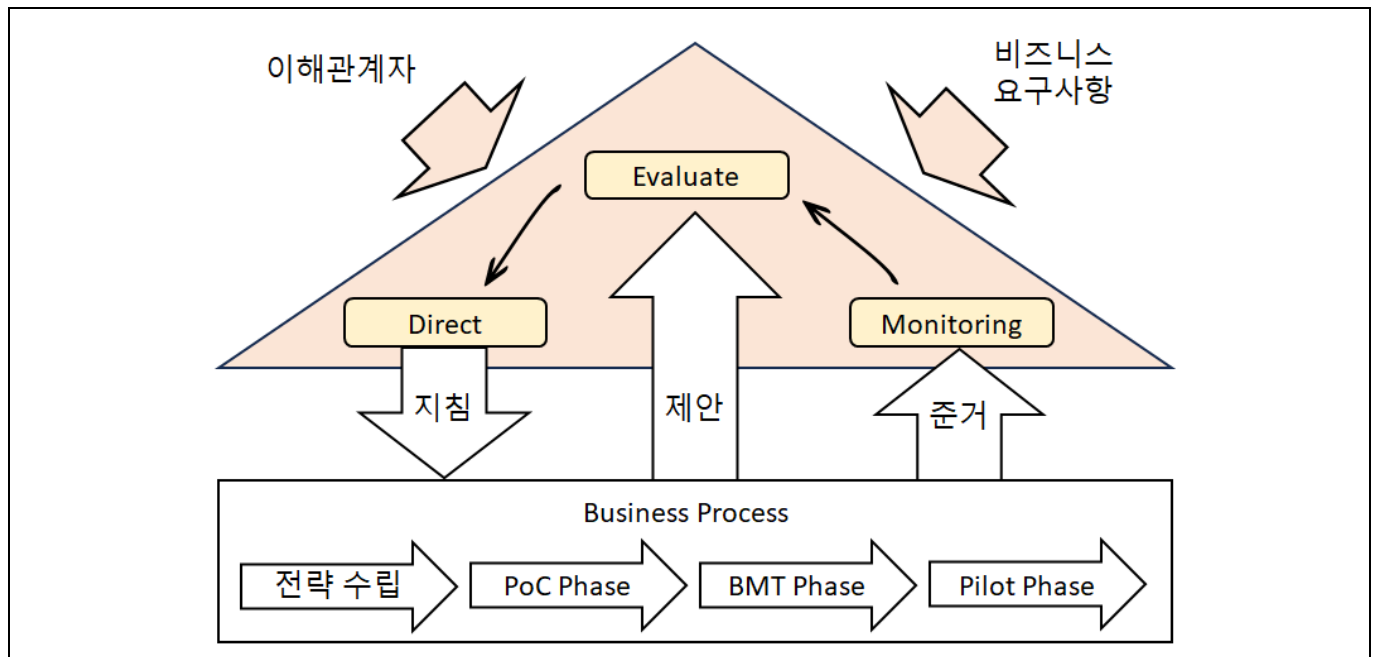
- 기획단계부터 사전에 PoC와 Benchmark Test를 수행하여, 제품을 선정하고 최종 운영 배포 전 Pilot Test를 수행하여 성공적인 프로젝트 완료가 가능

나. 프로젝트 수행 시 통합 적용 시나리오 상세

단계	적용 기법	시나리오 설명
기획 단계	- PoC	- 국내 도입 사례가 적은 시나리오 또는 고난이도의 커스터마이징이 필요한 영역의 프로토타이핑 진행 - 기술 타당성 검증으로 프로젝트 수행의 기술적 근거 확보가 목적
솔루션 선정 단계	- Benchmark Test	- 운영환경과 유사한 워크로드를 모델링하여 공통 지표(응답시간, 처리량)를 설정 후 진행 - 최적의 TCO(Total Cost of Ownership) 솔루션 선정
프로젝트 수행 단계	- N/A	- 프로젝트 발주 후 분석, 설계, 개발, 테스트 진행
구축 및 배포 단계	- Pilot Test	- 선정된 솔루션을 특정 부서나 지점에만 적용 - 실제 운영 데이터를 적용하여 업무 흐름 검증 - 현장 수용성 확인 및 운영 시 시행착오 최소화

- PoC 로 실현 가능한지 검증하고, BMT 로 최고의 제품을 선정하고, Pilot Test 로 실제 운영 중 발생 가능한 오류를 제거하는 것이 시나리오의 핵심

IV. PoC, Benchmark Test, Pilot Test 통합 연계를 위한 거버넌스 체계 구축

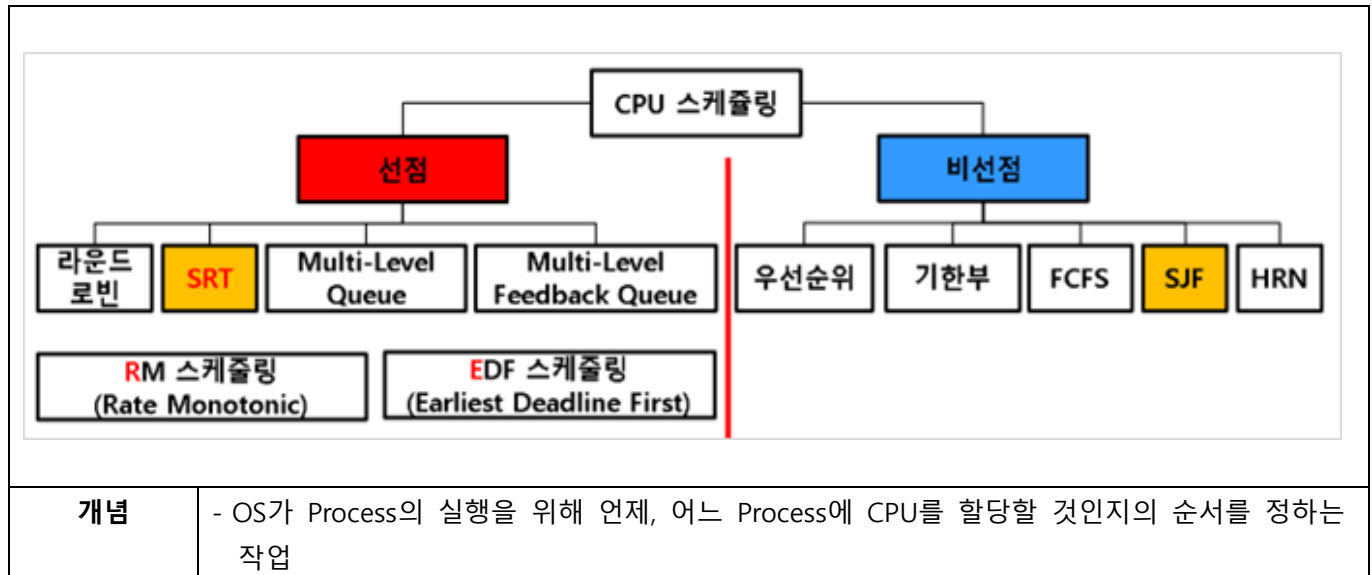


- 각 단계의 의사결정 주체와 산출물, 다음 단계로 연계 기준을 거버넌스 체계로 관리 및 통합 파이프라인 구축하여 프로젝트 수행 전 사전에 의사결정 수행 및 프로젝트 이행, 피드백 순환구조 구축으로 프로젝트 완전성 확보

“끝”

05	운영체제 스케줄링 기법		
문제	운영체제의 프로세스 스케줄링에 대하여 다음 사항을 설명 하시오. 가. 목적 나. 주요 스케줄링 알고리즘		
도메인	CA/OS	난이도	중 (상/중/하)
키워드	선점, 비선점, 라운드로빈, SRT, MLQ, MLFQ, RM, EDF, 우선순위, 기한부, FCFS, SJF, HRN		
출제배경	운영체제 스케줄링의 기본 지식 확인		
참고문헌	ITPE 기술사회 자료집		
출제자	정상반 정상 기술사(제 124회 정보관리기술사 / itpe_peak@naver.com)		

I. 자원을 효율적으로 배분하기 위한 기법, 운영체제 프로세스 스케줄링 개념



III. 운영체제 프로세스 스케줄링의 목적

가. 시스템 관점 스케줄링의 목적

항목	목적	설명
자원 이용률	CPU 이용률 극대화	- CPU가 단 1ms도 쉬지 않고 명령어를 처리하도록 하여, 비싼 하드웨어 자원의 가동률을 100%에 가깝게 유지
	균형 있는 장치 사용	- 특정 자원(CPU 또는 I/O)에만 작업이 몰리지 않도록 배분하여, 시스템 전체 부품이 골고루 일하게 함으로써 병목 현상을 방지
처리량	단위 시간당 작업 수 증대	- 1초 또는 1분 동안 완료되는 프로세스의 개수를 늘려, 시스템이 처리할 수 있는 전체 업무량을 최대화
	스케줄링 오버헤드 최소화	- 프로세스를 교체(Context Switch)할 때 발생하는 운영체제의 계산 비용을 줄여, 실제 작업에 투입되는 '순수 실행 시간'을 확보

- 시스템 관점외 사용자 관점에서도 프로세스 스케줄링의 목적 존재

나. 사용자 관점 스케줄링의 목적

항목	목적	설명
응답성	응답 시간 단축	- 사용자가 마우스를 클릭하거나 키보드를 쳤을 때, 시스템이 반응을 시작할 때까지 걸리는 시간을 줄임
	반환시간(Turnaround) 최적화	- 프로세스가 제출된 시점부터 실행을 마치고 최종 결과가 사용자에게 전달될 때까지의 전 과정을 빠르게 마무리
공정성	기아 현상(Starvation) 방지	- 우선순위가 낮은 프로세스라도 무한정 기다리지 않도록 배려하여, 모든 작업이 언젠가는 반드시 실행될 수 있도록 보장
	대기 시간의 평균화	- 특정 사용자나 특정 작업만 혜택을 보는 것이 아니라, 모든 프로세스가 준비 큐(Ready Queue)에서 기다리는 시간을 공평하게 관리

- 다양한 관점에서 목적이 존재하고 이 목적을 만족시키기 위해 스케줄링이 필요

III. 주요 스케줄링 알고리즘 설명

가. 선점형 스케줄링 알고리즘 설명

구분	정의	특징
SRT (Shortest Remaining Time)	- CPU자원의 효율성 극대화를 위해 계속해서 서비스 시간이 적은 작업을 파악하여 우선적으로 CPU를 할당하여 처리하는 선점형 스케줄링 기법 - 수행시간이 가장 짧다고 판단되는 작업을 먼저 수행	- 현재 실행중인 프로세스의 남은 서비스 기간과 준비 상태 큐에 새로 도착한 프로세스의 실행시간을 비교하여 가장 짧은 실행시간을 요구하는 프로세스에게 CPU할당 - 시분할 시스템에 활용 시 유용 - SJF에 비해 평균 대기시간이나 반환시간에서 효율적 - 준비상태 큐에 있는 각 프로세스의 서비스시간을 지속적으로 추적해야 하므로 오버헤드 증가
MLQ (Multi-Level Queue)	- 성격이 다른 여러 개의 그룹으로 분리하고, 각 큐마다 서로 다른 스케줄링 알고리즘을 적용하는 방식	- 개별 알고리즘 적용 : 각 큐는 독립적인 스케줄링 알고리즘을 가짐 - 큐 간의 우선순위 존재 : 각 큐 사이에는 고정된 우선순위가 있어, 상위 큐가 비어 있지 않으면 하위 큐의 프로세스는 실행될 수 없음 - 프로세스 고정 배치 : 한 번 특정 큐에 할당된 프로세스는 실행이 끝날 때까지 다른 큐로 이동할 수 없음
MLFQ (Multi-Level Feedback Queue)	- 프로세스가 실행되는 도중에도 큐 사이를 이동할 수 있는 스케줄링 기법	- 큐 간 이동 허용 : 프로세스가 할당된 시간(Time Quantum) 내에 작업을 못 마치면 우선순위가 낮은 아래쪽 큐로 강등 - 작업 성격 자동 분류 : I/O 위주 작업(짧은 실행)은 상위 큐에 머물게 하고, CPU 위주 작업(긴 실행)은 자연스럽게 하위 큐로 내려 보냄 - 에이징(Aging) 적용 : 낮은 우선순위 큐에서 너무 오래 대기한 프로세스를 주기적으로 상위 큐로 격상(Boosting)시켜 실행을 보장

나. 비선점형 스케줄링 알고리즘 설명

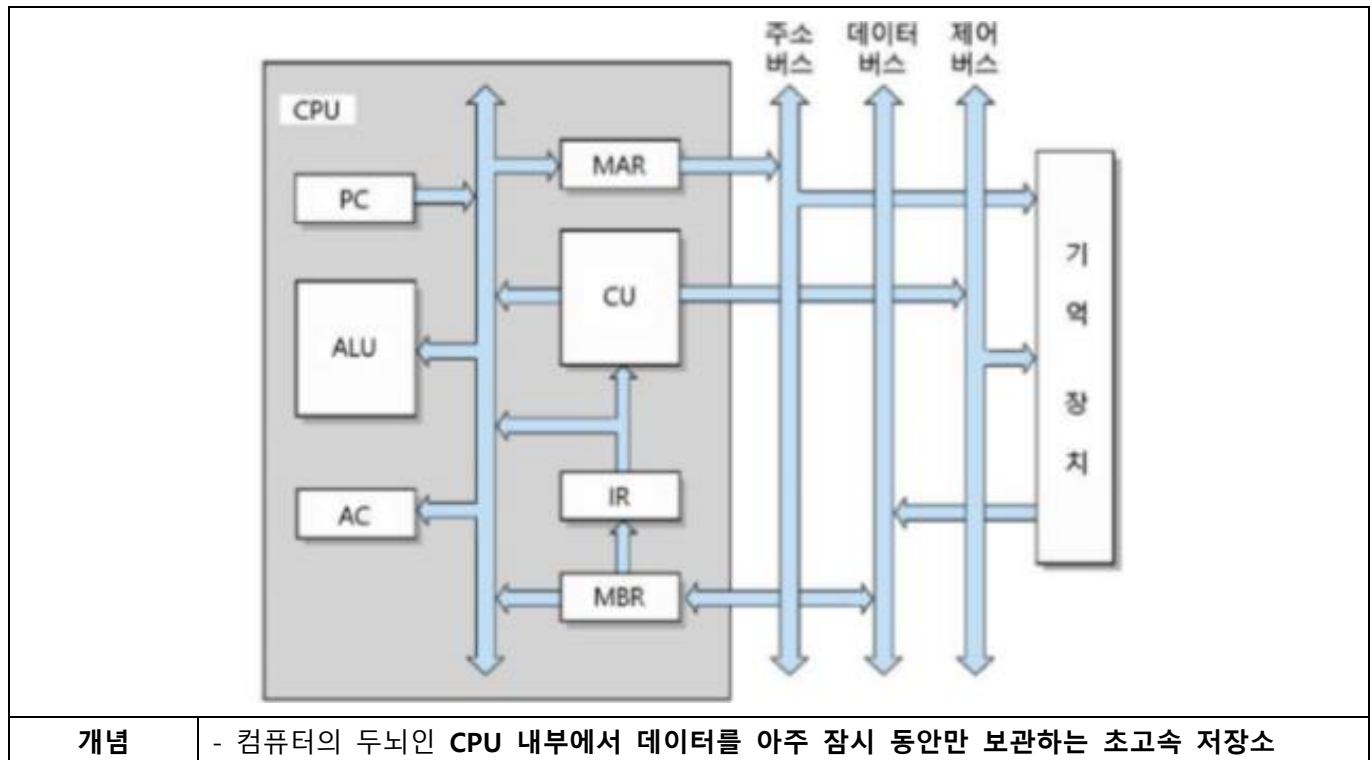
구분	정의	특징
SJF (Shortest Job First)	- 수행시간이 가장 짧다고 판단되는 작업을 먼저 수행하는 비선점 스케줄링 알고리즘	- Ready Queue에서 실행시간이 가장 짧은 프로세스를 선택하여 CPU할당 - 일괄처리 환경에서 구현이 용이함 - 작업시간이 적은 프로세스가 계속 들어오는 경우, 기존 작업시간이 긴 프로세스는 기아현상 발생 - SRT에 비해 평균 대기시간이 김 - SJF 처리시 기아현상(Starvation)의 발생 가능성이 있어 HRN을 통해 개선함
HRN (Highest Response-ratio Next)	- 응답률(Response Ratio)이라는 지표를 계산하여 그 값이 가장 높은 프로세스에게 CPU를 할당하는 방식	- 가변 우선순위 방식 : 대기 시간이 길어질수록 분자 값이 커져 응답률이 높아지므로, 우선순위가 유동적으로 변함 - 에이징 기법 적용 : SJF(최단 작업 우선)의 단점인 기아 현상(Starvation)을 해결하여, 긴 작업도 오래 기다리면 CPU를 할당 - 비선점형 : 한 번 CPU를 차지한 프로세스는 작업이 완료될 때까지 강제로 뺏기지 않고 실행을 보장
우선순위	- 각 프로세스마다 우선순위 번호(Priority Number)를 부여하고, 시스템은 이 번호를 기준으로 CPU를 배분	- 다양한 우선순위 : 시간 제한, 메모리 요구량, 프로세스의 중요도, 사용료 등 내부적 또는 외부적 요인에 의해 우선순위가 결정 - 선점 및 비선점 가능 : 새 프로세스가 현재 실행 중인 것보다 우선순위가 높을 때 CPU를 뺏으면 선점형, 끝날 때까지 기다리면 비선점형이 됨 - 기아현상 : 우선순위가 높은 작업이 계속 들어올 경우, 우선순위가 낮은 작업은 무한정 대기하게 되는 현상이 발생

- 운영체제 프로세스는 선점, 비선점형 알고리즘 중에서 상황에 따라 적용

“끝”

06	CPU 레지스터		
문제	명령어를 실행하기 위해 필요한 CPU 내부 레지스터 종류들에 대하여 설명하시오.		
도메인	CA/OS	난이도	중 (상/중/하)
키워드	PC, MAR, MBR, WR, GR/GPR, Base Register, Index Register, Operation code, Operand		
출제배경	CPU 구조에서 기본 레지스터 지식 확인		
참고문헌	ITPE 기술사회 자료집		
출제자	정상반 정상 기술사(제 124회 정보관리기술사 / itpe_peak@naver.com)		

I. 명령어 실행을 위한 필수 요소, CPU 레지스터 개념



III. 특수 전달, 데이터 레지스터 설명

가. 특수 전달 레지스터 설명

구분	주요 역할	설명
PC	프로그램의 순서를 카운트하는 역할	- 어떤 명령어가 몇번째 순서에 실행되는지를 기록하는 역할 - 다음에 실행될 명령어의 메모리 주소를 기억 - 다른 이름으로 IC(Instruction Counter) 또는 LC(Location Counter)라고도 부름
MAR	메모리 주소록 역할	- 데이터의 위치의 확인 - 비어있는 메모리 주소 확인
MBR	임시적 저장 공간	- 메모리에 저장되기 전 임시적으로 저장되는 공간 - MAR 에 메모리 주소를 요청

IR	명령어 기억	- Decoder 에 들어가기 전에 명령어를 기억하는 역할 - 명령어의 op-code 를 확인하여 어떤 연산을 수행하는지 찾음
----	--------	---

- 가장 중요한 레지스터로, 용도(역할)가 정해져 있는 레지스터

나. 데이터 레지스터 설명

구분	주요 역할	설명
MBR	프로그램의 순서를 카운트하는 역할	- 어떤 명령어가 몇번째 순서에 실행되는지를 기록하는 역할 - 다음에 실행될 명령어의 메모리 주소를 기억 - 다른 이름으로 IC(Instruction Counter) 또는 LC(Location Counter)라고도 부름
WR	메모리 주소록 역할	- 데이터의 위치의 확인 - 비어있는 메모리 주소 확인
GR/GPR	다목적 사용	- 다목적으로 사용되는 레지스터 - 여러가지 레지스터의 집합 의미 - ALU 와 연결되어 있지 않음

- 주기억장치에서 가져온 데이터를 저장하는 레지스터로 가져온 데이터의 크기와 동일한 크기를 할당함

III. 주소 레지스터와 그 외 레지스터 설명

가. 주소 레지스터 설명

구분	주요 역할	설명
Base Register	기준 주소 저장	- 프로그램이나 데이터 세그먼트의 시작 주소(기준점)를 보관 - 프로그램이 메모리의 어느 위치에 로드되더라도 이 값을 기준으로 상대적인 위치를 찾게 해줌
Index Register	주소 범위 및 수정	- 기준 주소로부터 얼마나 떨어져 있는지에 대한 변위(Offset) 값을 저장 - 주로 배열(Array) 처리나 반복문 내에서 데이터의 주소를 순차적으로 변경할 때 사용

- 주기억장치에 접근하기 위해서, 주기억장치의 주소 일부 또는 전부를 기억하는 레지스터

나. 그 외 레지스터 설명

구분	주요 역할	설명
Operation Code	수행할 동작 지정	- CPU 가 실행할 명령의 종류(더하기, 저장, 로드 등)를 나타내는 부분 - 명령어의 앞부분에 위치하며, 비트 수에 따라 수행 가능한 명령의 개수가 결정
Operand	연산 대상 지정	- 명령어가 실행될 때 필요한 데이터 자체나 데이터가 저장된 주소

		- 연산 코드에 의해 처리될 '재료' 역할을 수행하며, 주소 지정 방식에 따라 그 의미가 달라짐
Decoder	명령어 해석 및 분배	- IR(명령어 레지스터)에 있는 연산 코드를 받아 해당 명령이 무엇인지 분석 - 분석 결과에 따라 제어 장치(CU)를 통해 각 하드웨어 장치에 실행 신호를 보냄
Status Register	연산 결과 상태 기록	- ALU 에서 연산이 수행된 직후, 결과값이 0 인지(Zero), 음수인지(Negative), 오버플로우가 발생했는지 등의 상태(Flag)를 비트 단위로 저장하여 다음 명령에 참조

- 명령어가 인출된 이후, 각 레지스터는 순서에 따라 상호작용하여 CPU 를 움직임

“끝”



ITPE 기술사회

제138회 컴퓨터시스템응용기술사 기출문제

대 상	정보관리기술사, 컴퓨터시스템응용기술사, 정보통신기술사, 정보시스템감리사 시험
발행일	2026년 02월 07일
집 필	강정배PE, 정상PE, 전일PE, 백현PE, 조종흥PE
출 판	ITPE(Information Technology Professional Engineer)
주 소	ITPE 대치점 서울시 강남구 선릉로 86길 17 선릉엠티빌딩 7층 ITPE 선릉점 서울시 강남구 선릉로 86길 15, 3층 IT교육센터 아이티피이 ITPE 강남점 서울시 강남구 테헤란로 52길 21 파라다이스벤처타워 3층 303호 ITPE 영등포점 서울시 영등포구 당산동2가 하나비즈타워 7층 ITPE ITPE 을지로점 서울시 중구 삼일대로 363, 2615호(장교동 장교빌딩)
연락처	070-4077-1267 / itpe@itpe.co.kr

본 저작물은 [ITPE\(아이티피이\)](#)에 저작권이 있습니다.

저작권자의 허락없이 **본 저작물을 불법적인 복제 및 유통, 배포**하는 경우
법적인 처벌을 받을 수 있습니다.